



# Lesson 9: Neural Networks

Technologies for Artificial Intelligence

Fabio Antonini, Università degli Studi  
dell'Aquila

# Agenda

- A boundary whose **shape** is learned, not chosen in advance
- One neuron is one line, and exactly where that runs out
- Forward propagation, and backpropagation derived right to left
- Softmax, activations, and the vanishing gradient
- Initialisation, optimisers, regularisation: the failure modes

# Exercise 8, before we start

- What counts is **what the metric could see**, not the score itself
- The gap to watch for: a silhouette score taken as proof that a clustering is right, when it only rewards round, well-separated groups
- Today the same obligation returns in a new form: a network will reach a high training accuracy very quickly, and that number on its own says almost nothing

# What a network adds

- Boundaries so far had a shape **fixed in advance**
- A line; axis-parallel boxes; a curve from a chosen kernel
- A network learns the **shape of the boundary itself**
- Everything else today makes that possible, or stops it going wrong
- No architecture tour, no hardware, no network call

# Three problems, and a ceiling of 0.97

- **Acceptance test:** two axes, one pass/fail verdict
- **Two-channel drift:** the exclusive-or (XOR) function
- **Handwritten digits:** 1,797 images, ten classes, real
- The rig is **wrong on 3% of units**, so every score today is read against a ceiling of **0.97**
- A line scores **0.55** on acceptance and **0.93** on digits

# The perceptron, and a unit you already have

- Rosenblatt, 1958: weights, a bias, “output 1 if the sum is positive”
- Swap the threshold for a sigmoid: lesson 4’s logistic unit, `LogisticRegression`
- What is new: the unit becomes a **component** of a larger network
- Undecided at 0.5 means the weighted sum is zero
- A line in two dimensions, a hyperplane in general

A neuron is a line

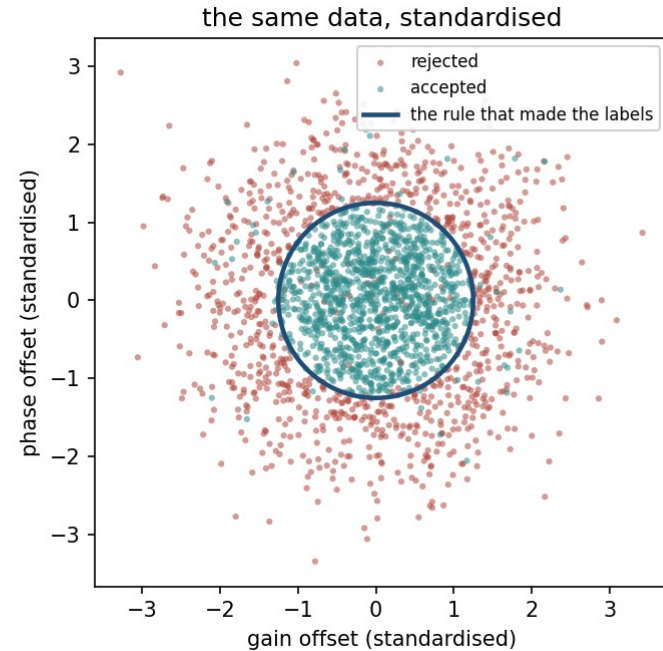
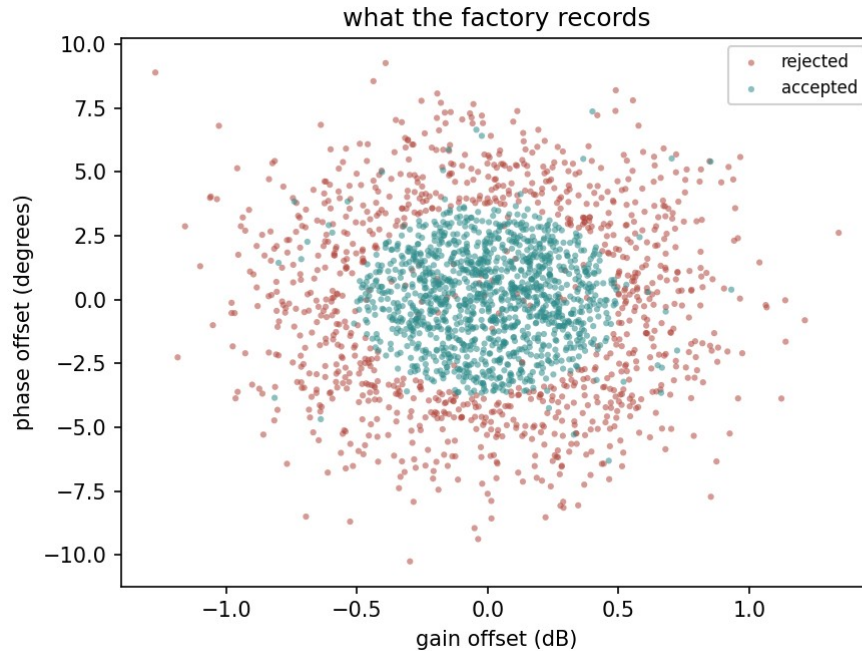
$$\hat{y} = \sigma(w^\top x + b), \quad \sigma(z) = \frac{1}{1 + e^{-z}}$$

# Meridian's acceptance test

- Two calibration axes: a gain offset in decibels, a phase offset in degrees
- It passes when both offsets are **jointly** small enough for one global firmware correction
- 3,000 sensors: `train_test_split` keeps 2,250 to train, then `StandardScaler`
- So the accept region is the inside of a **closed curve**
- And no straight line encloses anything



# 2,250 sensors, raw and standardised



# A circle of radius 1.25

- Gain tolerance **0.50**, production spread (standard deviation) **0.40**
- Phase tolerance **3.75**, production spread **3.00**
- Both are exactly **1.25 spreads**, so standardised, the accept region is a circle of radius 1.25
- Two units, two tolerances, one shared shape
- **0.5497** of the 3,000 generated units fall inside it

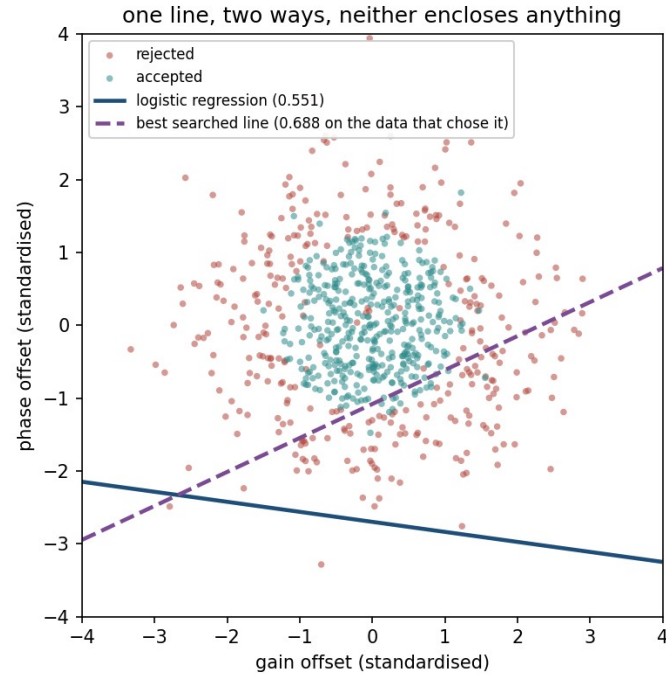
What fraction of units the circle holds

$$P(\|z\| < 1.25) = 1 - e^{-0.78125} = 0.5422$$

# What one line is worth

|                                            | test accuracy |
|--------------------------------------------|---------------|
| always predict “accepted”                  | 0.5480        |
| fitted logistic regression                 | <b>0.5507</b> |
| the best straight line for this population | 0.6491        |
| the rig’s ceiling                          | 0.9700        |

# No line encloses it



# The fit did not fail

- Coefficients (**0.0096, 0.0701**), intercept **0.1892**
- Almost exactly the constant model: no line does much better than that
- Not an optimiser that gave up. The reason is symmetry
- The accept region is a disc on the origin; every accepted unit has a partner opposite
- Any tilt gaining accuracy on one side gives it back on the other

# 72,762 candidate lines, and 4 points of self-deception

- The search tries 72,762 boundaries and scores the winner **on the data that chose it**: 0.6880
- The honest figure, computed exactly from the geometry: **0.6491**
- **Almost 4 points of pure selection bias**
- Nothing was fitted. No parameter estimated. Only a choice made
- Lesson 5's argument, inside a lesson-9 experiment

# The smallest problem that needs a hidden layer

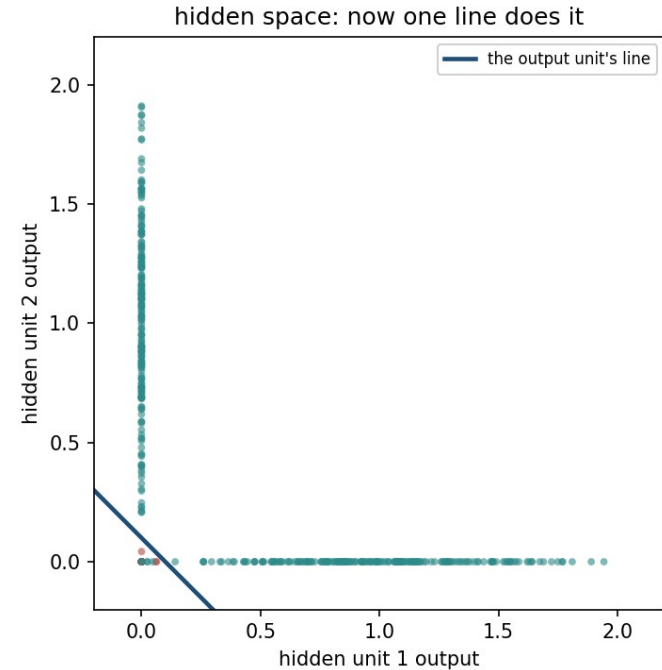
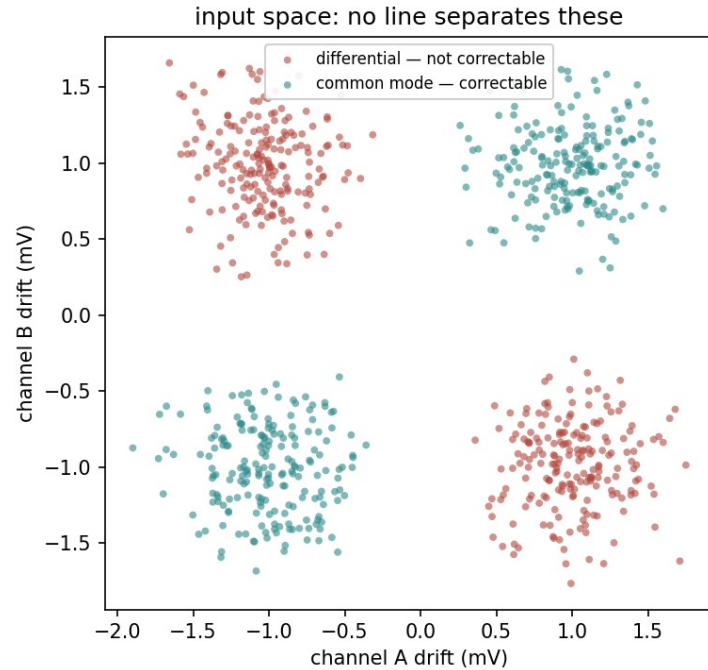
- One correction removes the drift only when both channels drift **the same way** (same sign)
- That is the exclusive-or (XOR) function
- Four clouds of 200 units; a line scores exactly **0.5000**
- Coding the channels as  $\pm 1$ , the rule is about  $|a + b|$
- “Large in absolute value” is the outside of a strip: **two boundaries, not one**



# Two units, written down rather than trained

- Two rectified linear units (ReLU), each  $\max(0, \cdot)$  of the same sum
- One fires when  $a + b > 1$ , the other when  $a + b < -1$
- Between them they compute  $|a + b|$ , clipped
- Output weights 10 and 10, bias -1: the rule is  $|a + b| > 1.1$
- **Nothing trained**, and over all 800 units it scores **0.9938**

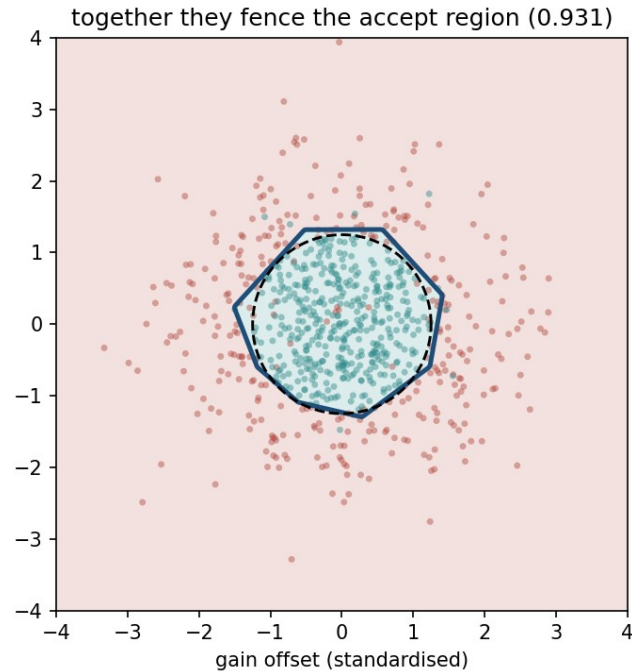
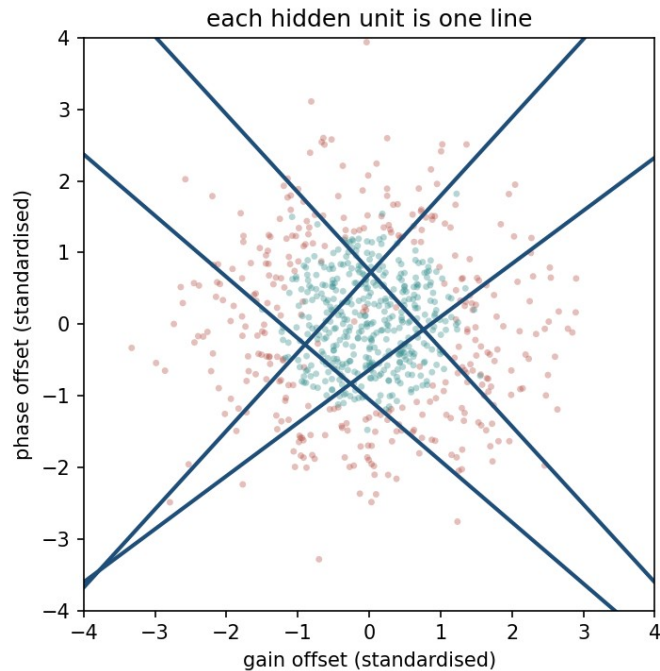
# What the hidden layer actually did



# The hidden layer classified nothing

- It **moved the data** until the output line was enough
- Correctable clouds to  $(1,0)$  and  $(0,1)$ ; the rest to the origin
- The name for this is **representation learning**
- What is learned: coordinates in which the boundary is simple
- Lesson 10's convolutional networks: the same trick, with most weights fixed at zero

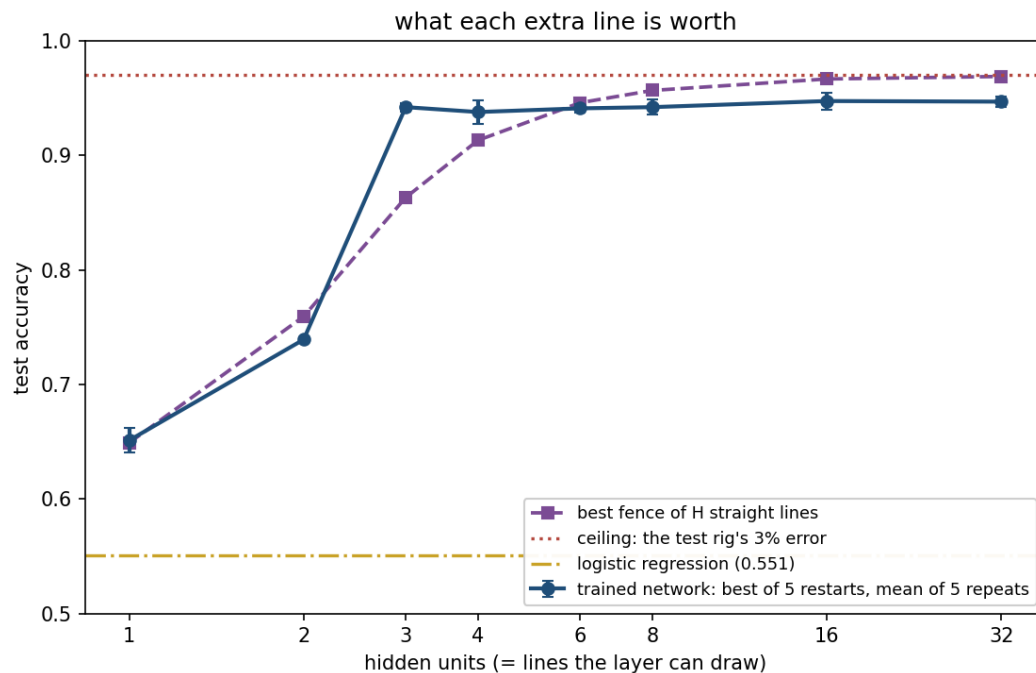
# Four lines enclose a region; one never does



# Capacity is not the same as findability

| hidden units | median accuracy | best of 20 | runs above 0.95 |
|--------------|-----------------|------------|-----------------|
| 2            | 0.7500          | 0.9975     | <b>4 of 20</b>  |
| 3            | 1.0000          | 1.0000     | 15 of 20        |
| 4            | 1.0000          | 1.0000     | 19 of 20        |
| 8            | 1.0000          | 1.0000     | 20 of 20        |

# What each extra line is actually worth



# Universal approximation, and what it does not promise

- Cybenko (1989), Hornik (1991): one hidden layer, enough units, any continuous function
- Read the quantifiers. It says such a network **exists**
- Nothing about how many units “enough” is
- Nothing about whether training will find it
- Nothing about behaviour outside the region
- Two units sufficed on the drift data: descent missed it 16 times in 20

# Forward propagation: examples in rows

- One hidden layer of  $H$  units, one output unit,  $m$  examples,  $n$  inputs
- Two matrix multiplies, each followed by an elementwise function
- Examples in **rows**, as scikit-learn and Keras; in columns every transpose flips
- In code: `keras.Sequential` of a `keras.Input` and `layers.Dense` layers



Layer 1: a line per unit, then a non-linearity

$$Z^{[1]} = XW^{[1]} + b^{[1]}, \quad A^{[1]} = g(Z^{[1]})$$

Layer 2: one more line, then a probability

$$Z^{[2]} = A^{[1]}W^{[2]} + b^{[2]}, \quad \hat{y} = \sigma(Z^{[2]})$$

# The shapes

| symbol           | shape        | what it is                             |
|------------------|--------------|----------------------------------------|
| X                | $m \times n$ | the design matrix, one example per row |
| W, layer 1       | $n \times H$ | one column per hidden unit             |
| b, layer 1       | H            | broadcast across rows                  |
| Z and A, layer 1 | $m \times H$ | pre-activations and activations        |
| W, layer 2       | $H \times 1$ | the output unit's weights              |
| $\hat{y}$        | $m \times 1$ | one prediction per example             |

# Two facts the shapes hand you

- **Examples never mix:** the row index passes through untouched
- Nothing lets example 3 influence example 7, which is what makes mini-batching valid
- **Each column of the first weight matrix is one unit**
- Its weight vector, and the line that unit draws
- So  $H$  hidden units are  $H$  lines, and the output unit votes

An untrained network costs 0.6721;  
guessing costs 0.6931

$$J = -\frac{1}{m} \sum_{i=1}^m [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$$

# Break

- Twelve minutes

# Two bad ways to get every gradient

- We need the derivative of the cost for **every** weight and bias
- **Perturb and re-run**: one forward pass per parameter; for notebook 3's network of 301,066 parameters, 301,066 passes for one step
- **Differentiate symbolically**: the same sub-expressions, thousands of times
- Both correct. Both unusable

# Compute each repeated piece once, right to left

- Carry the cost's derivative for each layer's **pre-activations**
- From it: that layer's weight gradients **and** the layer below's, so one backward sweep gives every gradient
- **A weight's gradient is how wrong the layer above was, times what this weight contributed**
- Keras automates it with `tf.GradientTape`



The output layer collapses to  
prediction minus truth

$$\delta^{[2]} = \frac{1}{m} (\hat{y} - y)$$

# Why sigmoid and cross-entropy belong together

- The loss derivative carries a factor  $1$  over  $\hat{y}(1 - \hat{y})$
- The sigmoid's derivative **is**  $\hat{y}(1 - \hat{y})$
- They cancel exactly, leaving  $\hat{\mathbf{y}} - \mathbf{y}$
- Squared error keeps that factor, and it is near zero exactly when the network is **confidently wrong**

# One step down, through the activation

- Weight gradient = incoming activations, transposed, times the signal above
- To step down: multiply by the weight matrix above, then by the activation's derivative
- For the ReLU that derivative is an indicator: 1 where the pre-activation was positive
- A unit that contributed nothing forward passes nothing backward

# Backward through one layer

$$\delta^{[1]} = (\delta^{[2]}(W^{[2]})^\top) \odot g'(Z^{[1]})$$

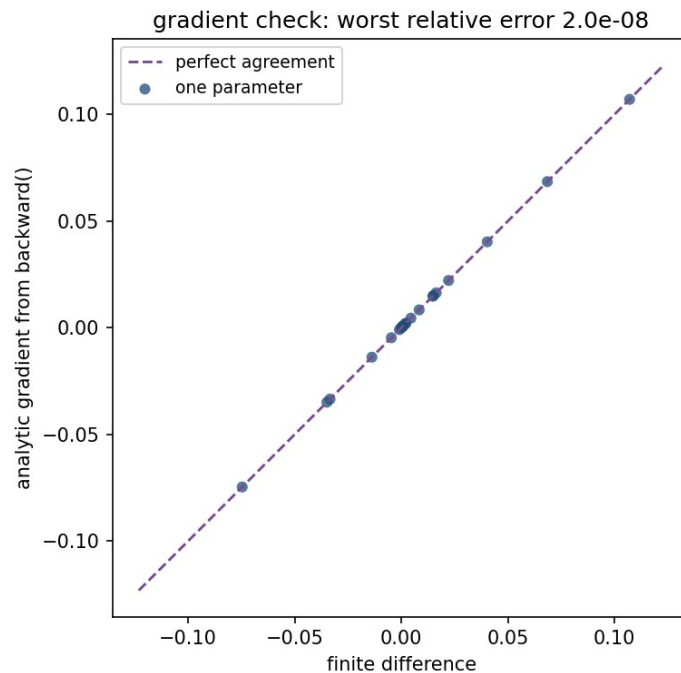
# What backpropagation costs

- The backward pass does about the same arithmetic as the forward pass
- **A gradient costs roughly what a prediction costs**: this is what makes training feasible at all
- But every layer's activations must be **stored**
- Memory grows with depth times batch size, and memory, not arithmetic, is what usually limits batch size

# Check the gradient before you trust it

- A wrong gradient **raises no exception**: it trains badly and looks like a modelling problem
- Compare each partial with  $(J(\theta + h) - J(\theta - h)) \div 2h$ : error of order  $h^2$
- Notebook 01: worst disagreement  $1.97 \times 10^{-8}$ , median  $7.83 \times 10^{-11}$
- **Relative error below  $10^{-6}$ : believe it. Above  $10^{-4}$ : there is a bug**
- Four lines of code, and not optional

# 21 partial derivatives, on the diagonal



# Notebook 1, live

| section, and its slides                  | you call (→ the number to come back with)                                                                 |
|------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| split, then one straight line, 8-15      | <code>train_test_split</code> , <code>StandardScaler</code> ,<br><code>LogisticRegression</code> → 0.5507 |
| forward, backward, gradient check, 24-39 | NumPy only → untrained cost 0.6721, worst disagreement $1.97 \times 10^{-8}$                              |
| the drift problem, 16-21                 | the hand-built network, then 20 restarts per width → 4 of 20 above 0.95 at two units                      |
| the polygon yardstick, 22                | <code>scipy.integrate</code> → the best 4-sided fence scores 0.9396                                       |
| zero initialisation, 52-53               | all weights 0 → final cost 0.6887                                                                         |



# Ten classes need ten outputs

- **Softmax** turns  $K$  scores into a probability distribution
- For  $K = 2$  it is the sigmoid: one construction, two sizes
- Loss, categorical cross-entropy:  $-\log(\text{probability of the true class})$ ,  
`sparse_categorical_crossentropy`
- The gradient at the output is again  $\hat{\mathbf{y}} - \mathbf{y}$ : the backward pass is unchanged

# Softmax

$$\text{softmax}(z)_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$$

# A hidden layer is not always the answer

| architecture                   | parameters | validation accuracy | sd over 5 seeds |
|--------------------------------|------------|---------------------|-----------------|
| softmax alone, no hidden layer | 650        | 0.9324              | 0.0035          |
| one hidden layer of 32         | 2,410      | 0.9602              | 0.0052          |
| one hidden layer of 64         | 4,810      | 0.9611              | 0.0045          |
| two hidden layers of 64        | 8,970      | 0.9676              | 0.0057          |

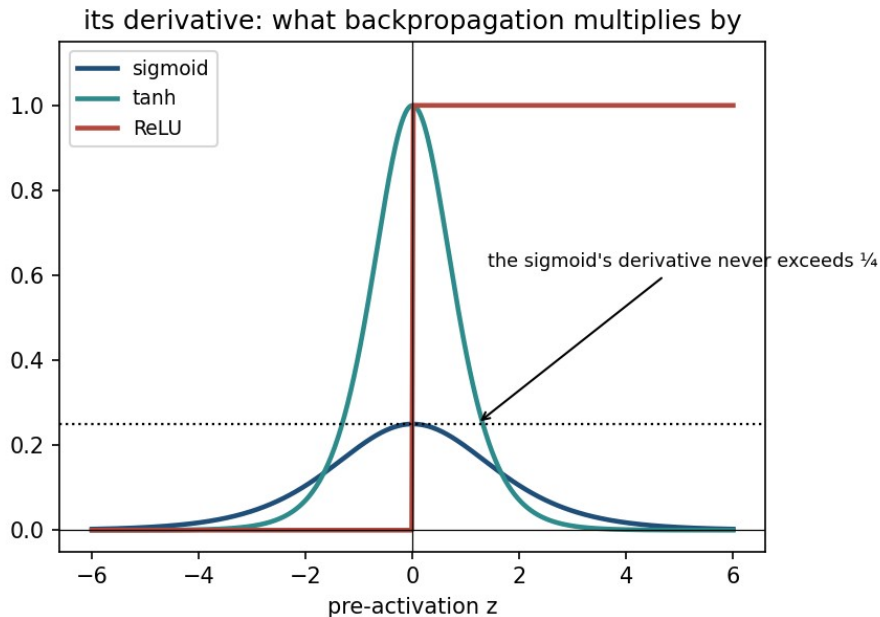
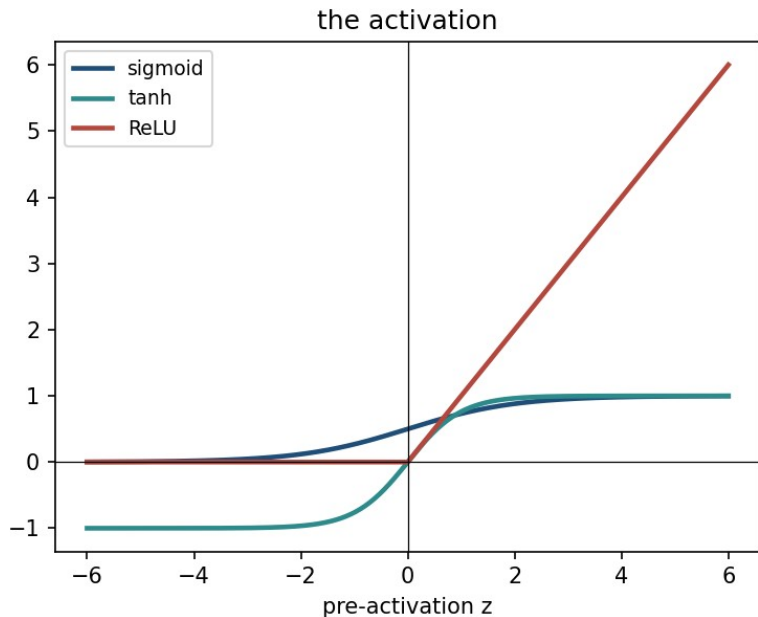
# Reading that table

- No hidden layer at all is within **3.5 points** of the best network here
- The first hidden layer is worth **2.8 points**; 32 units to 64, **0.09**
- On the acceptance data the same step was worth **39 points**
- Digits vote pixel by pixel: the acceptance rule needed two measurements **at once**

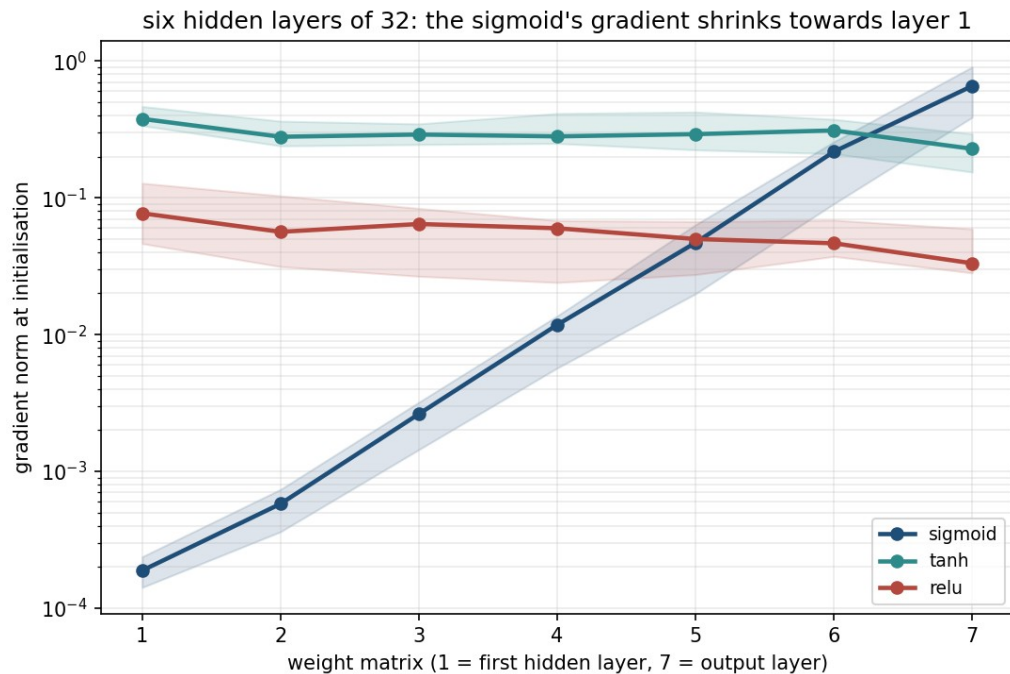
# A sigmoid layer divides the gradient by about four

- Any number of linear layers composes to one: the activation is the whole reason depth buys anything
- The sigmoid's derivative  $\sigma(z)(1 - \sigma(z))$  peaks at  $\frac{1}{4}$ , at zero only
- Backpropagation multiplies by it **once per layer**
- At initialisation the weights multiply it by about 1, so the derivative decides: **each sigmoid layer divides the gradient by about four**

# Three activations, and their derivatives



# Six layers of it, measured



# One draw is not a law

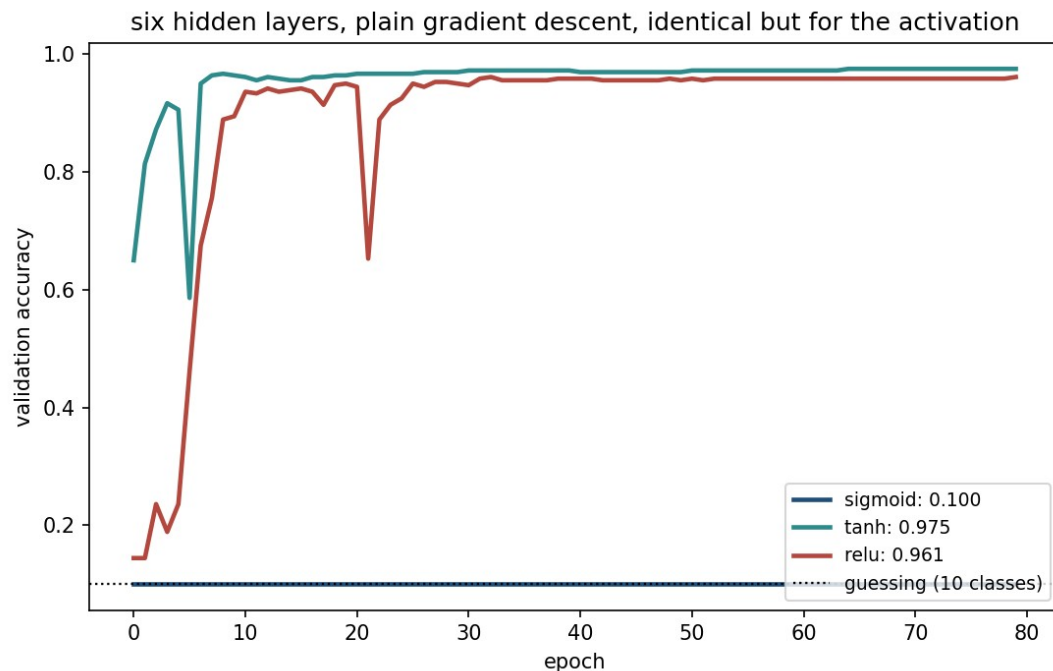
- Gradient at the last weight matrix  $\div$  gradient at the first: median **3,547**
- Over the same eight seeds it ranges **2,734 to 4,607**
- Quoting the largest reports one draw as a law
- **The per-layer factor of about 4 is the property**
- By depth: 9.5, 170, 3,553, 46,250 at two, four, six and eight layers



# Eighty epochs: only one of them never learns

- Same six layers, same data, differing **only** in the activation
- Sigmoid: **0.1000**, exact chance on ten classes
- tanh: **0.9750**, past 0.85 within 3 epochs
- Rectified linear unit: **0.9611**, 9 epochs to get going
- What kills the sigmoid is not squashing: it is *where its derivative is bounded*

# The sigmoid never leaves chance



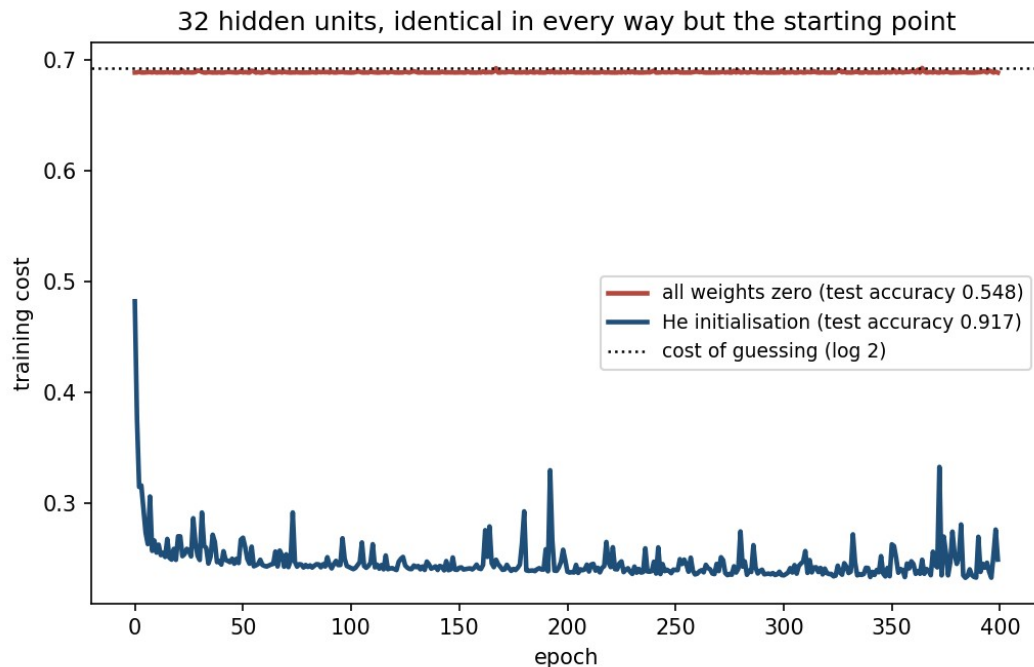
# Notebook 2, live

| section, and its slides           | you call (→ the number to come back with)                                                                                  |
|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| the same network in Keras, 24-29  | <code>keras.Sequential, keras.Input, layers.Dense, SGD</code> ; <b>then</b> compile, fit, evaluate → <b>0.9333</b>         |
| ten classes, 41-44                | <code>Dense(10, activation="softmax"), Adam, Dropout, confusion_matrix</code> → <b>0.9676</b> with <b>two layers of 64</b> |
| the shrinkage, measured, 45-48    | <code>tf.GradientTape, sparse_categorical_crossentropy</code> → <b>about 4 per sigmoid layer</b>                           |
| training six layers, 49-50        | SGD on three activations → sigmoid 0.1000, tanh 0.9750                                                                     |
| how big the weights should be, 54 | <code>RandomNormal</code> at two scales, against Glorot → <b>73% saturated at <math>N(0, 1)</math></b>                     |

# Zero cannot work

- Lesson 3 started from zero, and was right to: those costs are convex
- All-zero weights **freeze** a network: every gradient is exactly zero
- **Only the output bias ever moves**
- The bias converges to the base rate, and the cost to the cost of always predicting it, the label entropy: **0.6887 predicted, 0.6887 measured**
- Distinct hidden columns: **1 of 32**

# Only the starting point differs



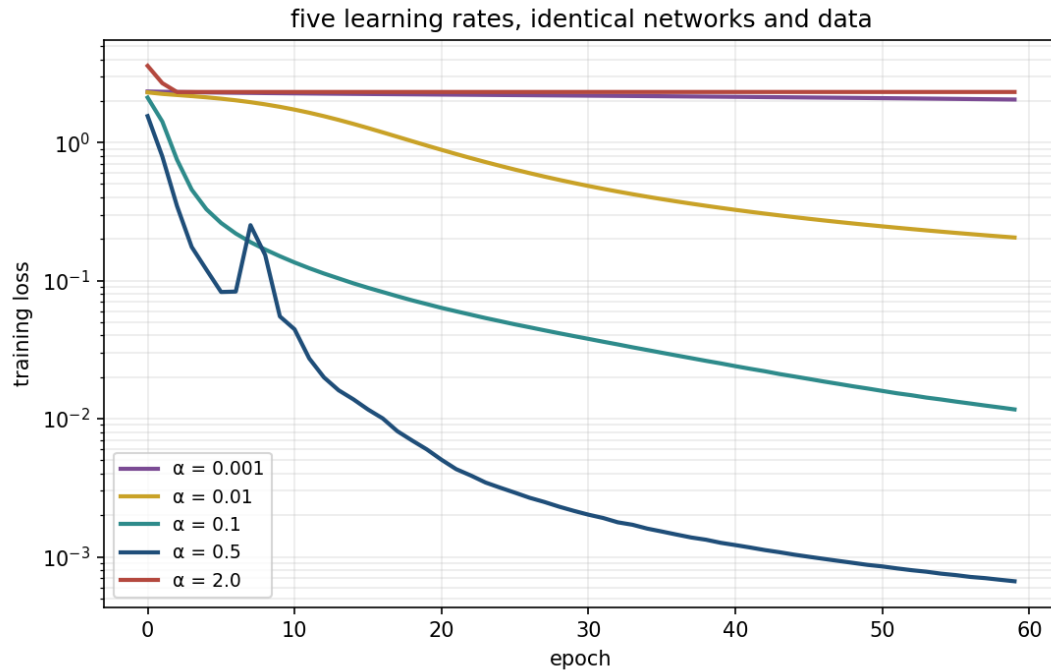
# How large? Propagate the variance

- A unit sums  $n$  inputs, so variance is multiplied by  $n \times \text{Var}(w)$  each layer
- Set  $\text{Var}(w) = 1/n$ : **Glorot**. ReLU halves it, so it wants  $2/n$ : **He**
- **Too small collapses**: indistinguishable from zero by layer 4
- **Too large saturates** rather than exploding: 73% of the last layer's tanh units past  $|a| > 0.99$
- Glorot loses half its spread over eight layers

# The learning rate, swept

| $\alpha$ | final training loss | validation accuracy | sd over 5 seeds |
|----------|---------------------|---------------------|-----------------|
| 0.001    | 1.9550              | 0.5574              | 0.0794          |
| 0.01     | 0.1965              | 0.9398              | 0.0035          |
| 0.1      | 0.0115              | 0.9556              | 0.0039          |
| 0.5      | 0.0007              | <b>0.9685</b>       | 0.0026          |
| 2.0      | 2.3222              | 0.1009              | 0.0013          |

# Three regimes, and a narrow band between them





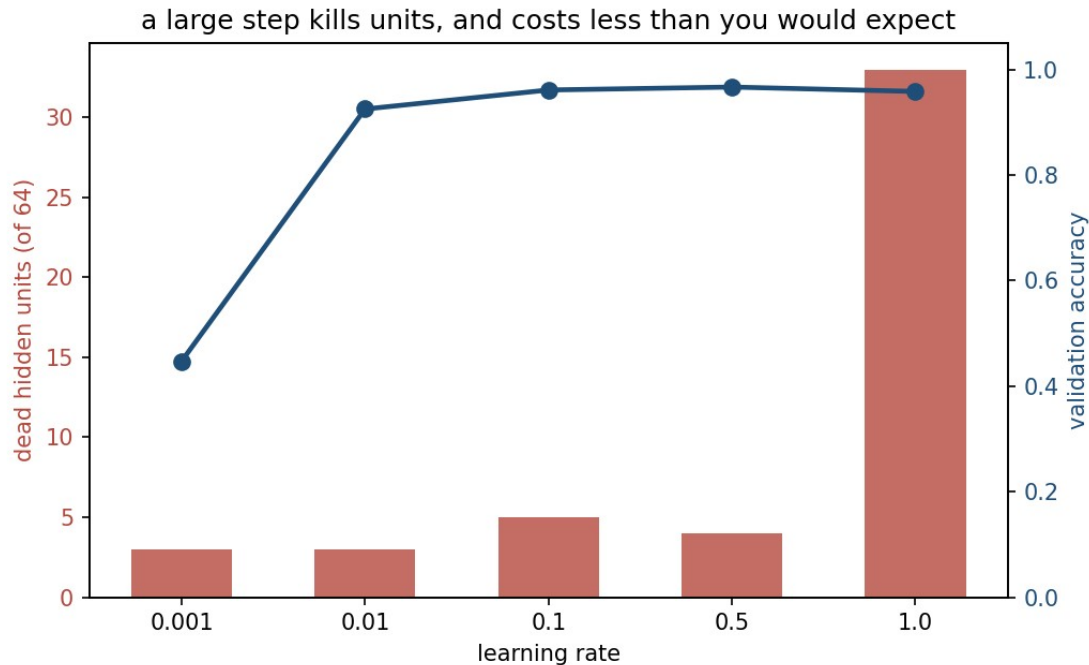
# The predictable mistake

- Too small, and the loss is still falling when the epochs run out
- **It looks exactly like a network that is too small:** both accuracies low
- The instinct that follows is sound: low training accuracy really is the signature of underfitting
- **Vary  $\alpha$  over orders of magnitude before touching the architecture**

# The rectified linear unit has its own failure

- Its derivative is exactly 1 on the active side: so, no vanishing gradient
- But a unit negative for **every** example gets zero gradient, for ever
- **Dead**: the gradient that would revive it is the one it cannot receive
- At  $\alpha = 1.0$ , **33 of 64 are dead**: accuracy 0.9583 against a best of 0.9667

# Half the layer dead, and almost no visible cost



# Mini-batches, momentum, and Adam

- Mini-batch **stochastic gradient descent (SGD)**, SGD: a small batch's gradient, noisy but right on average
- **Momentum**: step along a running average of past gradients
- **Adam (adaptive moment estimation)**, Adam: each parameter's step scaled by its own recent gradients, so a badly chosen  $\alpha$  matters less

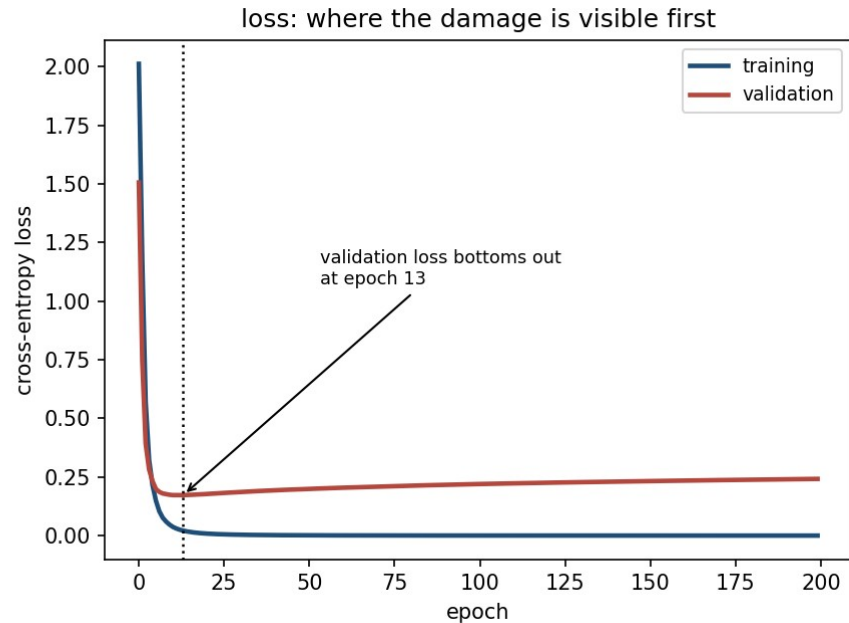
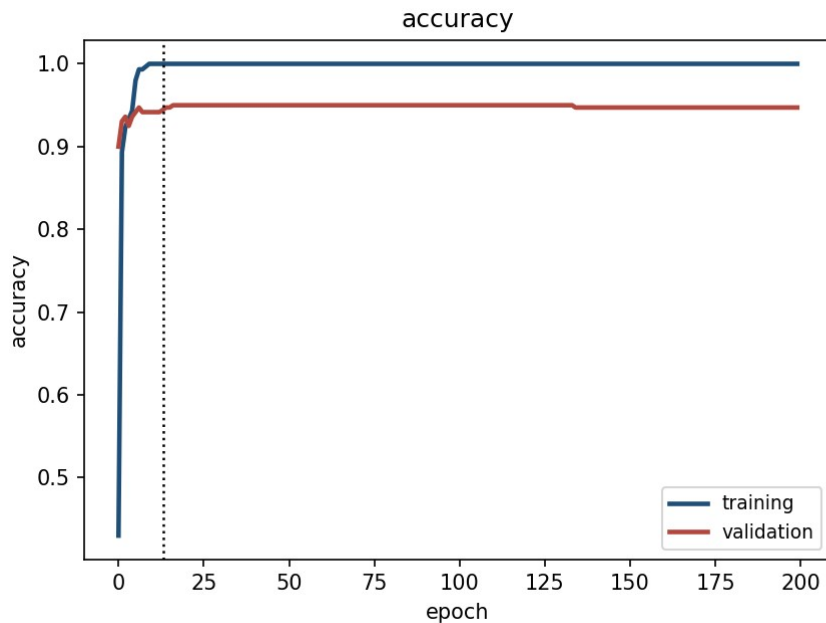
# Adam, measured: read the spread before the mean

| optimiser              | test accuracy | sd over 5 seeds | worst of 5 | vs the true rule |
|------------------------|---------------|-----------------|------------|------------------|
| plain gradient descent | 0.9389        | 0.0184          | 0.9027     | 0.9685           |
| with momentum 0.9      | 0.9349        | 0.0094          | 0.9173     | 0.9645           |
| Adam                   | <b>0.9485</b> | 0.0072          | 0.9360     | 0.9792           |

# Three regularisers, and what they bought

| method                                                      | test accuracy | sd over 5 seeds | train – test gap | epochs run |
|-------------------------------------------------------------|---------------|-----------------|------------------|------------|
| early stopping only<br>(EarlyStopping: keep the best epoch) | 0.9411        | 0.0069          | 0.0589           | 38.8       |
| + dropout 0.4<br>(Dropout: drop 40% of units per batch)     | 0.9483        | 0.0038          | 0.0503           | 48.8       |
| + L2 $10^{-3}$ (l2: a penalty on squared weights, as Ridge) | 0.9478        | 0.0011          | 0.0522           | 198.4      |
| + dropout and L2                                            | <b>0.9511</b> | 0.0045          | 0.0489           | 161.6      |

# It memorises the training set, and generalises anyway



# Notebook 3, live

| section, and its slides  | you call (→ the number to come back with)                                                         |
|--------------------------|---------------------------------------------------------------------------------------------------|
| the learning rate, 55-57 | <code>SGD(learning_rate=<math>\alpha</math>)</code> at five rates → best 0.9685 at $\alpha = 0.5$ |
| dead units, 58-59        | <code>keras.Model</code> to read the hidden layer → 33 of 64 dead at $\alpha = 1.0$               |
| optimisers, 60-61        | <code>SGD</code> , <code>SGD(momentum=0.9)</code> , <code>Adam</code> → Adam's worst run 0.9360   |
| memorising, 63           | two layers of 512 on 300 digits, <code>count_params</code> → 301,066 parameters                   |
| regularisers, 62         | <code>EarlyStopping</code> , <code>Dropout(0.4)</code> , <code>l2(1e-3)</code> → 0.9411 to 0.9511 |
| more data                | <code>train_test_split</code> at six sizes → +3.2 points from 300 to 1,077 examples               |



# What to take away

- **A neuron is a line:** three enclose a region, one never does
- A hidden layer **moves the data**; it does not classify
- **Backpropagation is the chain rule** - so gradient-check it
- **A sigmoid layer divides the gradient by about four**
- Report a difference with its spread

# Homework

- **Exercise 9**, discussed at the start of **Friday 4 December**
- `Exercises/09_neural_networks.md`